

VITyarthi Flipped Course Project

STUDENT STUDY & ASSIGNMENT TRACKER

Submitted by: [YOUR NAME]

Registration Number: [YOUR REGISTRATION NUMBER]

Course: [COURSE NAME / CODE]

Academic Year: 2026–27

Important: Replace all bracketed placeholders and add your own screenshots, testing observations, and any changes you actually made before submission.

1. Introduction

The Student Study & Assignment Tracker is a command-line Python application designed to help students organize academic tasks. It provides a simple way to store tasks, monitor completion, check deadlines and generate a progress summary.

2. Problem Statement

College students may have multiple assignments, practice activities, tests and deadlines at the same time. A simple task-management application can reduce confusion by keeping academic tasks in one place and showing their completion status and deadlines.

3. Objectives

The objectives are to: (1) provide a simple task-management system; (2) allow students to track completed and pending work; (3) identify upcoming and overdue deadlines; (4) calculate progress statistics; and (5) demonstrate modular Python programming, file handling, validation and testing.

4. Functional Requirements

Module 1 – Task Management: add, view, update, delete and complete tasks.

Module 2 – Progress Tracking: calculate total, completed and pending tasks and completion percentage, including subject-wise progress.

Module 3 – Deadline & Reporting: identify upcoming/overdue tasks and generate a summary report.

5. Non-Functional Requirements

Usability: the application uses a simple numbered command-line menu.

Reliability: task data is stored in a JSON file so it can be loaded again.

Error Handling: invalid menu choices, task IDs, empty fields and dates are checked.

Resource Efficiency: the application uses lightweight local JSON storage and Python standard-library modules.

6. System Architecture

The system follows a simple modular architecture. The user interacts with **main.py**. Task operations are handled by **task_manager.py**, progress calculations by **progress_tracker.py**, deadlines by **deadline_manager.py**, input checking by **input_validator.py**, and reporting by **report_generator.py**. Task data is stored in **data/tasks.json**.

7. Workflow Diagram

START → Display Menu → User Selects Operation → Validate Input → Perform Operation → Save/Read JSON Data → Display Result → Return to Menu → Exit → END

8. Use Case Diagram

Actor: Student

The Student can: Add Task, View Tasks, Update Task, Delete Task, Complete Task, View Progress, View Deadlines and Generate Report.

9. Sequence Diagram

For adding a task:

Student → main.py → task_manager.py → input_validator.py → tasks.json → main.py → Student.

10. Component/Class Design

Main components are: Main Controller, Task Manager, Progress Tracker, Deadline Manager, Input Validator, Report Generator and JSON Data Store. The modules communicate through Python functions and shared task data.

11. Data/Storage Design

The project uses a JSON list. Each task contains: **id**, **subject**, **task**, **deadline** and **status**.

12. Design Decisions & Rationale

Python was selected because it is the course programming language and supports rapid development. JSON was selected because it is simple, human-readable and sufficient for a small local student application. The program is divided into modules so that task management, validation, progress, deadlines and reporting can be maintained separately.

13. Implementation Details

The application uses functions, conditional statements, loops, lists, dictionaries, file handling, JSON serialization, date processing, input validation and modular imports. The program is started with **python main.py**.

14. Screenshots / Results

Insert your own screenshots here after running the project. Recommended screenshots: main menu, adding a task, task list, progress result, deadline result, report result and an invalid-input/error-handling example.

15. Testing Approach

Test cases should verify valid task creation, invalid dates, invalid task IDs, task completion, progress calculation and empty-data handling. Record the actual result after running each test.

16. Challenges Faced

[Write the actual challenges you faced while developing the project, such as debugging input validation, handling JSON data, organizing modules or testing functions.]

17. Learnings & Key Takeaways

[Write what you personally learned. Suggested areas to discuss: functions, modules, lists/dictionaries, file handling, JSON, validation, testing and Git/GitHub.]

18. Future Enhancements

Possible future improvements include a graphical interface, user accounts, database storage, recurring tasks, reminders, charts and cloud synchronization.

19. References

VITyarthi Build Your Own Project – General Project Instructions & Submission Guidelines. Python documentation may also be consulted for language/library details used in the implementation.